IS Audit & Control

Abdul-Rahman Mahmood
Assistant Professor, Computer Science, FAST-NU

abdulrahman@nu.edu.pk
alphapeeler.sf.net/pubkeys/pkey.htm
pk.linkedin.com/in/armahmood
bqb-tsid-asp
alphapeeler
alphapeeler
armahmood786
http://alphapeeler.sf.net/
reddit.com/user/alphapeeler
www.flickr.com/alphapeeler
http://alphapeeler.tumblr.com
armahmood786@jabber.org
alphapeeler@aim.com
abdulmahmood-sss
alphapeeler@icloud.com
pinterest.com/alphapeeler

Auditing Databases

Background

- **(RDBMS). Database management systems (DBMS) maintain data records and their relationships, or indexes, in tables. Relationships can be created and maintained across and among the data and tables.**
- **Database: any collection of data in any structured form. For instance, a flat file that contains customer records can serve as a database for an application.**

Database Auditing Essentials

- Client-server model - Early 1990s:
  - Desktop program connecting over a network directly to a DB backend. Referred to as a 2-tier application.
- 3-tiered applications - In the late 1990s:
  - Consisted of a web browser connecting to a middle-tier web application. The middle tier then connected to the DB backend. Custom software didn't need to be installed on every client workstation, and software updates could be applied to a central server. Clients could run any operating system that supported a basic browser. Securing the database was much simpler. The danger now exists that an attacker will circumvent the web application to attack the backend database.

Common Database Vendors

- **Oracle**
  - Flavors: Standard Edition, Enterprise Edition, OracleLite, Express Edition.
  - Branched out: Berkeley DB (Sleepycat) open-source, embedded database, MySQL (Sun Microsystems), The TimesTen (In-Memory Database), InnoDB, a transaction engine for the MySQL database.
- IBM
  - DB2 Universal DB (AIX, Linux, HP-UX, Sun / Windows)
  - DB2 Universal DB for z/OS (mainframe)
  - Informix Dynamic Server: for legacy applications.
  - Information Management System (IMS), since 1969, Hierarchical DB. IMS typically runs on the mainframe . Does not usually work in a client-server model.

Common Database Vendors

- MySQL
  - open-source DB used extensively in small or medium-sized web applications. Under GNU Public License by MySQL AB, a Swedish company. MySQL has a large and growing grassroots : LAMP (Linux, Apache, MySQL, and PHP) open-source web platform. MySQL AB was purchased by Sun in February 2008, and Sun was later purchased by Oracle in 2010, making MySQL an Oracle product. MySQL has been a bare-bones database, providing a small fraction of the functionality available from other database vendors. Administration costs are relatively low, and provides performance for all but the most demanding web applications. MySQL 5.0 has stored procedures, views, and triggers. Simplest databases to secure from hacking.

Common Database Vendors

- MaxDB:
  - MySQL AB also offers a second open-source database called MaxDB, which is designed specifically as a high-reliability backend for SAP systems.
  - Sybase Sybase:
    - acquired by SAP in 2010. Sybase produces several DBs:
      - The flagship Sybase Adaptive Server Enterprise, database, designed for enterprise databases.
      - Sybase Adaptive Server Anywhere, designed as a lighter-weight database
    - Sybase originally partnered with Microsoft to develop the early versions of its database system, which was referred to at the time as Sybase SQL Server on Unix and Microsoft SQL Server on Windows.

Common Database Vendors

- As of version 4.9, Microsoft and Sybase split the code line and went their separate ways. Sybase has expanded beyond databases as well. The company offers various developer tools and a web application server and currently is focused on the delivery of data to mobile devices. Although the company has lost significant market share to the competition in the database market, it continues to maintain a presence in many places, and its databases will continue to exist for a long time.

Common Database Vendors

- Microsoft: MSSQL Server is one of the most popular databases owing to its low price tag and its simplistic administration model. It has several flavors:
  - MSSQL Server 7.0 is an older version of the product with a few legacy installations still in existence.
  - MSSQL Server 2000 (a.k.a. SQL Server 8.0) was Microsoft's main database version for five years. As such, it is heavily entrenched in a large number of enterprises.
  - MSSQL Server 2005 provided a rich new set of security features among other functionality over its predecessor.
  - MSSQL Server 2008 is the latest in Microsoft's line and continues to have a wide adoption through its strong integration with other Microsoft products.

Common Database Vendors

- MS Database Engine (MSDE) is a free version of SQL Server providing a backend for independent software vendors (ISVs) to embed databases in their applications. Because MSDE is free, it is embedded in a large number of applications and is very common. With the delivery of SQL Server 2005, MSDE has been renamed to SQL Server 2005 Express Edition.
- **PostgreSQL**
  - **Postgres**, is an object-relational database management system (ORDBMS) with an emphasis on extensibility. It can handle workloads ranging from small single-machine applications to large Internet-facing applications with many concurrent users.
  - 31st March 2016: PostgreSQL 9.5.2

General Database Components

- Program File
- Configuration Values
- Data Files
- Client/Network Libraries
- Backup/Restore System
- SQL Statements
- Database Objects
- Data Dictionary

Database Components

- **DB objects in modern RDBMS:**
  - Table: Stores rows of data in one or more columns.
  - View: A SELECT statement on top of a table or another view that creates a virtual table. Views can change the number or order of columns, can call functions, and can manipulate data in a variety of ways.
  - Stored procedure/function Procedural code: called to execute complex functionality within the DB. Functions return values. Procedures do not return values. Stored procedures are very efficient for data access.
  - Trigger: Procedural code, called when table is modified. Perform actions, including modifications to tables.
  - Index Mechanism to provide fast lookup of data. Indexes are complex objects, and their proper tuning is critical to database performance.

Test Steps for Auditing Databases

Setup and General Controls

Test Steps for Auditing Databases

- 1. Obtain the database version and compare with your corporate policy requirements. Verify that the database is running a database software version the vendor continues to support.
- 2. Verify that policies and procedures are in place to identify when a patch is available and to apply the patch. Ensure that all approved patches are installed per your database management policy.
- 3. Determine whether a standard build is available for new database systems and whether that baseline has adequate security settings.

Test Steps for Auditing Databases

Operating System Security

Test Steps for Auditing Databases

- 4. Ensure that access to the operating system is properly restricted.
- 5. Ensure that permissions on the directory in which the database is installed, and the database files themselves, are properly restricted.
  - Ensure that the "Everyone" or "Anonymous" user does not have any permissions on database files.
  - drives that store database files must use NTFS.
- 6. Ensure that permissions on the registry keys used by the database are properly restricted.
  - Review the security permissions through the Registry Editor, through a command-line utility such as GetDACL.

Test Steps for Auditing Databases

Account and Permissions Management

Test Steps for Auditing Databases

- 7. Review and evaluate procedures for creating user accounts and ensuring that accounts are created only with a legitimate business need. Also review and evaluate processes for ensuring that user accounts are removed or disabled in a timely fashion in the event of termination or job change.
- 8. Check for default usernames and passwords.
- 9. Check for easily guessed passwords.

Test Steps for Auditing Databases

- Table classifies these default usernames and passwords into a few categories. Literally thousands of these default passwords can be found on various security websites.

Category	Description
Default database password	Created in a standard database install. Can depend on the installed components of the database. Most of the latest versions of databases have eliminated default database passwords, but default passwords continue to be a serious concern in older versions of database software.
Sample or example passwords	Many samples, examples, and demonstrations of new or existing features are shown in SQL scripts that include creation of a test or sample account.
Default application password	When you install third-party products on top of a database, the products often install and run using a default username and password to access the database. These are known to hackers and serve as a common access route.
User-defined default password	When a new account is created, the password is often set to an initial value and then reset on first use. Problems arise when an account is created but never accessed. Ensure that passwords set on new accounts are random, strong passwords.

Test Steps for Auditing Databases

- 10. Check that password management capabilities are enabled.
  - Many of the database platforms provide support for rich password management features. Oracle leads this area by including capabilities for the following features:
  - • Password strength validation functions
  - • Password expiration
  - • Password reuse limits
  - • Password expiration grace time
  - • Password lockout
  - • Password lockout reset

Test Steps for Auditing Databases

- 11. Verify that DB permissions are granted or revoked appropriately for the required level of authorization.
  - Database privileges are slightly different from operating system permissions. Privileges are managed using GRANT and REVOKE statements. For instance, the following SQL statement gives USER1 the permission to SELECT from the SALARY table:
    - `GRANT SELECT ON SALARY TO USER1`
  - The REVOKE statement is used to remove permissions that have been granted:
    - `REVOKE SELECT ON SALARY FROM USER1`
  - The GRANT statement can be used selectively to give permissions, such as SELECT, UPDATE,DELETE, or EXECUTE.

Test Steps for Auditing Databases

- 12. Review database permissions granted to individuals instead of groups or roles.
  - You should attempt to grant permissions to roles or groups, and those permissions, in turn, should be granted to individuals within those roles or groups.
- 13. Ensure that database permissions are not implicitly granted incorrectly.
  - Review the permission model for database platform and verify that permissions are inherited appropriately.
  - Review system privileges allowing access to data, e.g., **SELECT ANY TABLE** or granting a privileged role to user.
  - Document permissions that are implicitly as well as explicitly granted to ensure that permissions are not allowed when they are not appropriate.

Test Steps for Auditing Databases

- 14. Review dynamic SQL executed in stored procedures.
  - Running a stored procedure allows you to access objects as the stored procedure owner. This can be dangerous if stored procedures are not constructed properly
  - Restrict use of dynamic SQL in procedures that run with administrative privileges.
  - In MSSQL, A dynamically build Transact-SQL statements can be executed using **EXECUTE** Command or **sp_executesql** statement.

Test Steps for Auditing Databases

```
/* Using EXECUTE Command */
/* Build and Execute a Transact-SQL String with a single parameter
 value Using EXECUTE Command */
/* Variable Declaration */
DECLARE @EmpID AS SMALLINT
DECLARE @SQLQuery AS NVARCHAR(500)
/* set the parameter value */
SET @EmpID = 1001
/* Build Transact-SQL String with parameter value */
SET @SQLQuery = 'SELECT * FROM tblEmployees WHERE EmployeeID = ' +
CAST(@EmpID AS NVARCHAR(10))
/* Execute Transact-SQL String */
EXECUTE(@SQLQuery)
```

There are two variables declared. The first variable @EmpID is used as a parameter to the SQL Query and second Variable @SQLQuery is used to build the SQL String. You can clearly see that the variable @EmpID is cast to a NVarchar type and made as a part of the SQL String. If you print the @SQLQuery string (PRINT @SQLQuery), you will get the actual SQL query as shown below:

```
SELECT * FROM tblEmployees WHERE EmployeeID = 1001
```

Test Steps for Auditing Databases

- Example 2 - `Using sp_executesql`

```
/* Using sp_executesql */
/* Build and Execute a Transact-SQL String with a single parameter
value Using sp_executesql Command */

/* Variable Declaration */
DECLARE @EmpID AS SMALLINT
DECLARE @SQLQuery AS NVARCHAR(500)
DECLARE @ParameterDefinition AS NVARCHAR(100)
/* set the parameter value */
SET @EmpID = 1001
/* Build Transact-SQL String by including the parameter */
SET @SQLQuery = 'SELECT * FROM tblEmployees WHERE EmployeeID = @EmpID'
/* Specify Parameter Format */
SET @ParameterDefinition =  '@EmpID SMALLINT'
/* Execute Transact-SQL String */
EXECUTE sp_executesql @SQLQuery, @ParameterDefinition, @EmpID
```

you will get the query as shown below:

```
SELECT * FROM tblEmployees WHERE EmployeeID = @EmpID
```

Test Steps for Auditing Databases

- 15. Ensure that row-level access to table data is properly implemented
  - Unfortunately, DB's are not well designed to restrict access to a subset of rows in a table.
  - How:
    - Oracle offers virtual private databases (VPDs) that you can use to limit access to specific rows.
    - You also can use views programmatically to restrict rows returned based on the user's context.
    - A common and practical approach is to use stored procedures to access tables. Using this strategy, the DBA does not need to grant permissions on the table, preventing the user from attempting to circumvent the stored procedure.
- 16. Revoke PUBLIC permissions where not needed.
  - Many of the built-in stored procedures and functions in a database are granted to the PUBLIC group by default.

Test Steps for Auditing Databases

- 16. Revoke PUBLIC permissions where not needed.
  - Some built-in procedures & functions default to PUBLIC access, but blindly revoking all can be disastrous.
    - List all permissions, highlighting PUBLIC grants. Consult the DBA on current and future usage. Assess risks before revoking unused permissions. If agreed, revoke them. Always back up and create an undo script.
- 17. Verify that network encryption is implemented.
  - Network encryption protects credentials and data in transit. Networks are insecure—IP spoofing and traffic sniffing are risks. Encrypt traffic externally and internally.
    - Ensure network and client drivers support SSL encryption. Verify settings on both ends. Sample traffic if needed.

Test Steps for Auditing Databases

- 18. Verify that encryption of data at rest is implemented where appropriate.
  - Data at rest encryption secures stored data, crucial due to its long lifespan. Most data theft occurs from storage, not transit.
    - Ensure proper data encryption and secure key storage. Encryption is only as strong as its key protection. Avoid storing keys with encrypted data. Ensure the disaster recovery plan includes key management. Losing keys makes database recovery impossible.

Test Steps for Auditing Databases

- See details of following in the book.
- 19. Verify the appropriate use of database auditing and activity monitoring.
  - • Enabling native auditing in the database • Monitoring network traffic of audit database activity • Reviewing transaction logs to build an audit trail from the database
- 20. Evaluate how capacity is managed for the database environment to support existing and anticipated business requirements.
- 21. Evaluate how performance is managed and monitored for the database environment to support existing and anticipated business requirements.

Test Steps for Auditing Databases

- Tools and Technology

Database Auditing Tool	Website
AppDetective by Application Security, Inc.	www.appsecinc.com
NGSAuditor and NGSSquirrel by NGS Software, Ltd.	www.ngssoftware.com/home.aspx

Monitoring Tool	Website
DbProtect from Application Security, Inc.	www.appsecinc.com/products/dbprotect
Guardium	www.guardium.com
AuditDB from Lumigent	www.lumigent.com/solutions/database-auditdb.html
SecureSphere from Imperva	www.imperva.com/products/products.html

Data Encryption Tool	Website
Vormetric	www.vormetric.com
DbEncrypt from Application Security, Inc.	www.appsecinc.com/products/index.shtml
Defiance Security Suite from Protegrity	www.protegrity.com/DefianceSecuritySuite
Encryptionizer from NetLib	www.netlib.com
DataSecure from SafeNet	www.safenet-inc.com/Products/Data_Protection/Data_Encryption_and_Control/DataSecure.aspx

Test Steps for Auditing Databases

Resource	Website
Oracle Database Security Checklist, by Oracle Corporation	www.oracle.com/technology/deploy/security/database-security/pdf/twp_security_checklist_database.pdf
SANS Oracle Security Checklist	www.sans.org/score/oraclechecklist.php
Ten Steps to Securing SQL Server 2000	www.microsoft.com/sql/techinfo/administration/2000/security/securingsqlserver.asp
SQLSecurity.com Checklist	www.sqlsecurity.com
NIST Security Checklists	web.nvd.nist.gov/view/ncp/repository
DISA Checklists	iase.disa.mil/stigs/checklist/
ISACA Auditing Guidelines	www.isaca.org
Links to papers and presentations covering Oracle security	www.petefinnigan.com/orasec.htm
Oracle security website	www.oracle.com/technology/deploy/security/index.html
Research Team	**Website**
Argeniss Information Security	www.argeniss.com
Red-Database-Security	www.red-database-security.com
Application Security, Inc., Team SHATTER	www.appsecinc.com/aboutus/teamshatter/index.html
NGS Research	www.ngssoftware.com
Pentest Limited	www.pentest.co.uk
Pete Finnigan	www.petefinnigan.com
Integrigy	www.integrigy.com
Chip Andrews	www.sqlsecurity.com

- Master Checklist

❑	1. Obtain the database version and compare it against policy requirements. Verify that the database is running a version the vendor continues to support.
❑	2. Verify that policies and procedures are in place to identify when a patch is available and to apply the patch. Ensure that all approved patches are installed per your database management policy.
❑	3. Determine whether a standard build is available for new database systems and whether that baseline has adequate security settings.
❑	4. Ensure that access to the operating system is properly restricted.
❑	5. Ensure that permissions on the directory in which the database is installed, and the database files themselves, are properly restricted.
❑	6. Ensure that permissions on the registry keys used by the database are properly restricted.
❑	7. Review and evaluate procedures for creating user accounts and ensuring that accounts are created only when there's a legitimate business need. Also review and evaluate processes for ensuring that accounts are removed or disabled in a timely fashion in the event of termination or job change.
❑	8. Check for default usernames and passwords.
❑	9. Check for easily guessed passwords.
❑	10. Check that password management capabilities are enabled.
❑	11. Verify that database permissions are granted or revoked appropriately for the required level of authorization.
❑	12. Review database permissions granted to individuals instead of groups or roles.
❑	13. Ensure that database permissions are not implicitly granted incorrectly.
❑	14. Review dynamic SQL executed in stored procedures.
❑	15. Ensure that row-level access to table data is implemented properly.
❑	16. Revoke PUBLIC permissions where not needed.
❑	17. Verify that network encryption is implemented.
❑	18. Verify that encryption of data at rest is implemented where appropriate.
❑	19. Verify the appropriate use of database auditing and activity monitoring.
❑	20. Evaluate how capacity is managed for the database environment to support existing and anticipated business requirements.
❑	21. Evaluate how performance is managed and monitored for the database environment to support existing and anticipated business requirements.